7db1b1bd-b3ba-4064-a3f8-2ecb4a7f8faf

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'microsoft/mcp' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/mcp/mcp (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-03T19:38:42.477Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
DI Refactor: Azure.Mcp.Tools.ApplicationInsights
</pr_title>

<pr_description>
- [x] Inject `IApplicationInsightsService` into `RecommendationListCommand` constructor (remove `context.GetService<T>()`)
- [x] Update `RecommendationListCommandTests` to pass service mock directly to command constructor
- [x] Add shared `_commandDefinition` field to tests and use `_commandDefinition.Parse(...)` in test methods (Step 5)
- [x] Build and verify changes (2/2 tests pass)

<issue_title>DI Refactor: Azure.Mcp.Tools.ApplicationInsights</

issue_title>
><issue_description>## Summary
>
> Refactor `Azure.Mcp.Tools.ApplicationInsights` to use constructor dependency injection instead of resolving services via `context.GetService<T>()` in `ExecuteAsync`.
>
> This is a sub-issue of microsoft/mcp#158.
>
> ## Instructions
>
> Follow the algorithm described in `tools/di-refactor-plan.md` with `{ToolArea} = ApplicationInsights`.
>
> An example PR is: https://github.com/microsoft/mcp/pull/1815</issue_description>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@conniey</author>
@copilot+claude-sonnet-4.6 Verify that Step 5 was done correctly.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot+claude-sonnet-4.6 Verify that Step 5 was done correctly.

Step 5 (shared `_commandDefinition` field in tests) was missing. Fixed in 529b4b9 — added `private readonly Command _commandDefinition` initialized in the constructor, and updated both test methods to use `_commandDefinition.Parse(...)` instead

of `_command.GetCommand().Parse(...)`.

> [!WARNING]
>
>

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>tools/Azure.Mcp.Tools.ApplicationInsights/src/Commands/Recommendation/RecommendationListCommand.cs:15</file>
<original_commit>529b4b9</original_commit>

```
<comment_new>
<comment_id>2880118105</comment_id>
<author>@conniey</author>
@copilot Double-check that Step 5 was done correctly.
</comment_new>
```

</comment_thread>

</pr_reviews>

</comments>

The last **3** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment

and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<prev_assistant>

Fixed in `a1970c9`. The Step 5 test pattern was incomplete — the test had three inconsistencies vs the AppConfig reference pattern:

1. `_serviceMock` → renamed to `_applicationInsightsService` (service name convention)
2. `var logger = ...` (local variable) → `private readonly ILogger<RecommendationListCommand> _logger` (class field)
3. `new RecommendationListCommand(logger, _serviceMock)` → `new(_logger, _applicationInsightsService)` (implicit `new()` syntax)

All 2 tests still pass.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'microsoft/mcp' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/mcp/mcp (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-03T19:55:48.901Z</current_datetime>

Consider the following problem statement:

<problem_statement>

```
*This section includes details on the pull request with code
changes you have already submitted to fix the problem above.*

<pr_title>
DI Refactor: Azure.Mcp.Tools.ApplicationInsights
</pr_title>

<pr_description>
- [x] Inject `IApplicationInsightsService` into
`RecommendationListCommand` constructor (remove
`context.GetService<T>()`)
- [x] Update `RecommendationListCommandTests` to pass service
mock directly to command constructor
- [x] Add shared `_commandDefinition` field to tests and use
`_commandDefinition.Parse(...)` in test methods (Step 5)
- [x] Fix test to fully match DI refactor pattern: rename
`_serviceMock` → `_applicationInsightsService`, make logger a
class field, use implicit `new()` syntax (Step 5)
- [x] Build and verify changes (2/2 tests pass)

<issue_title>DI Refactor: Azure.Mcp.Tools.ApplicationInsights</
issue_title>
&gt;<issue_description>## Summary
&gt;
&gt; Refactor `Azure.Mcp.Tools.ApplicationInsights` to use
constructor dependency injection instead of resolving services
via `context.GetService<T>()` in `ExecuteAsync`.
&gt;
&gt; This is a sub-issue of microsoft/mcp#158.
&gt;
&gt; ## Instructions
&gt;
&gt; Follow the algorithm described in <a href="https://
github.com/microsoft/mcp/blob/main/tools/di-refactor-
plan.md">`tools/di-refactor-plan.md`</a> with `{ToolArea} =
ApplicationInsights`.
&gt;
&gt; An example PR is: https://github.com/microsoft/mcp/pull/
1815</issue_description>
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;</pr_description>

## Comments on the PR (you are @copilot in this section)
```

<comments>

<pr_comments>

<comment_new>
<comment_id>3986771421</comment_id>
<author>@conniey</author>
@copilot+claude-sonnet-4.6 Verify that Step 5 was done correctly.
</comment_new>

<comment_old>
<author>@copilot</author>

> @copilot+claude-sonnet-4.6 Verify that Step 5 was done correctly.

Step 5 (shared `_commandDefinition` field in tests) was missing. Fixed in 529b4b9 — added `private readonly Command _commandDefinition` initialized in the constructor, and updated both test methods to use `_commandDefinition.Parse(...)` instead of `_command.GetCommand().Parse(...)`.

> [!WARNING]
>
>

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>tools/Azure.Mcp.Tools.ApplicationInsights/src/Commands/Recommendation/RecommendationListCommand.cs:15</file>
<original_commit>529b4b9</original_commit>

<comment_old>
<author>@conniey</author>
@copilot Double-check that Step 5 was done correctly.
</comment_old>

<comment_old>
<author>@copilot</author>
The test was missing a few parts of the Step 5 pattern. Fixed in

a1970c9:
- Renamed `_serviceMock` → `_applicationInsightsService` (consistent with service name convention)
- Moved `logger` from local variable to `private readonly ILogger<RecommendationListCommand> _logger` class field
- Changed constructor call to use implicit `new(_logger, _applicationInsightsService)` syntax

The test now matches the same pattern as `AccountListCommandTests` (AppConfig).
</comment_old>

<comment_new>
<comment_id>2880118105</comment_id>
<author>@conniey</author>
@copilot Step 5 includes executing New-ChangeLogEntry.ps1 to generate CHANGELOG entry. This is not done.
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>